

Reviewer Pack — Minimal p-adic Valuation Rank and DPLL Search Tree Diameter ...

Entry 6842c23852ff · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 13:12:59 UTC

Minimal p-adic Valuation Rank and DPLL Search Tree Diameter Correlation

Entry ID: 6842c23852ff **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-01 13:12:59 UTC

1. Conjecture

Field A (mathematical branch): p-adic Valuation Theory **Field B** (complexity object): DPLL Search Trees

Statement:

For any given CNF φ with n variables, the minimal p-adic valuation rank ($\text{p-rank}(\varphi)$) of its associated clause set is linearly correlated with the diameter ($\text{diameter}(\varphi)$) of its DPLL search tree, such that $\text{p-rank}(\varphi) = \Theta(\text{diameter}(\varphi))$.

Rationale (proposer's reasoning):

The p-adic valuation ring provides a framework for studying algebraic properties in number theory. The correlation between p-rank and DPLL search tree diameter may reveal underlying structural similarities between arithmetic properties of clause sets and their computational complexity in resolution proof systems.

Taxonomy category: p-adic_valuation_dpll (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f3bd93844602f363

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For a CNF φ with n variables ($n \leq 40$), if the p-adic valuation rank ($\text{p-rank}(\varphi)$) is within ± 3 standard deviations of the expected mean diameter ($\text{diameter}(\varphi)$), then support for the conjecture is provided. Otherwise, the conjecture is falsified.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - p-adic valuation rank AND DPLL search tree diameter - DPLL search tree AND p-rank correlation - linear relationship p-adic valuation rank DPLL tree diameter

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def hensel_lifting(clause_set, p):
        # Placeholder implementation for Hensel lifting algorithm
        # This is a dummy function and should be replaced with actual logic
        return 0

    def dpll_search_tree_diameter(cnf):
        # Placeholder implementation for DPLL search tree diameter calculation
        # This is a dummy function and should be replaced with actual logic
        return 0

    n = random.randint(5, 40)
    cnf = [random.choice([1, -1]) * random.randint(1, n) for _ in range(n)]

    p_rank = hensel_lifting(cnf, 2)
    diameter = dpll_search_tree_diameter(cnf)

    return {
        "metric_name": "p-rank vs Diameter",
        "metric_value": p_rank,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(2, 97) for _ in range(30)]
```

```

results = []
for seed in seeds:
    result = run_trial(seed)
    print(f"TRIAL: {result}")
    results.append(result)

mean_value = sum(r["metric_value"] for r in results) / len(results)
std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

ic_value': 0, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 25, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 23, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 16, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 36, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 19, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}
TRIAL: {'metric_name': 'p-rank vs Diameter', 'metric_value': 0, 'instances_tested': 1, 'n_max': 17, 'conjecture_holds': False, 'counterexample': 'mapping_undefined'}

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ffec5f7.py", line 64, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9ffec5f7.py", line 64, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~~

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Safety rail: critic_challenge_falsified | original: The test results indicate that the conjecture does not hold for all tested instances, as 'conjecture_holds' is False for all trials. | next: Investigate the cause of the 'mapping_undefined' error and address it to ensure correct computation of p-rank and diameter. Additionally, consider increasing the number of test cases or exploring alternative methods to validate the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	33663
2	preregistration	ollama_remote	glm4:latest	0	0	9609
3	novelty	ollama_remote	glm4:latest	0	0	16582
4	novelty	ollama_remote	glm4:latest	0	0	16119
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20051
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12771
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8544
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14846
9	judge	ollama_remote	glm4:latest	0	0	30394

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 162579 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/6842c23852ff.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/6842c23852ff.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/6842c23852ff.tar.gz` (if generated)